請把我當成沒學過機器學習的人 從零開始講解LightGBM如何實作

可以。我會把你當成**完全沒學過機器學習**，從「LightGBM 到底在做什麼」開始，不假設你知道決策樹、梯度下降、Boosting 等概念。

我們可以把學習分成這條路線：

資料 → 機器學習 → 決策樹 → Boosting → Gradient Boosting → LightGBM → 實際寫 Python → 調參 → 模型解釋

---

1. 先不要管 LightGBM，它到底是什麼？

先想像你有一份資料：

年齡	收入	是否買房
25	30,000	否
32	50,000	否
35	80,000	是
42	100,000	是
28	70,000	否
45	120,000	是

我們希望訓練一個模型：

> 給它「年齡、收入」，讓它預測「是否買房」。

例如：

```
年齡 = 36
收入 = 85,000

        ↓

LightGBM

        ↓

買房機率 = 82%
```

這就是**機器學習（Machine Learning）**最基本的概念：

> 從過去的資料中學習規則，再拿學到的規則去預測新的資料。

---

2. 那 LightGBM 是什麼？

LightGBM 是 Microsoft 開發的一套 **Gradient Boosting Decision Tree（GBDT） 演算法/實作。

名稱可以拆成：

Light + GBM

其中 GBM = **Gradient Boosting Machine**。

所以你可以先記住：

> **LightGBM 本質上是一種「很多棵決策樹合作完成預測」的方法。**

這句話非常重要。

它不是神秘的黑盒子。

它的核心其實是：

```
決策樹
  ↓
決策樹
  ↓
決策樹
  ↓
決策樹
  ↓
很多棵樹一起預測
```

---

3. 什麼是決策樹？

先完全不談數學。

假設我要預測一個人會不會買房。

我可能先問：

```
收入 > 70,000？
       /     \
     是        否
     ↓         ↓
  年齡>35？    不買
   /   \
 是      否
 ↓       ↓
買      不確定
```

這就是一棵**決策樹（Decision Tree）**。

它一直問問題：

> 「某個特徵是否大於某個值？」

例如：

```
收入 > 70,000？
年齡 > 35？
工作年資 > 5？
房價 < 1,000 萬？
```

最後走到一個結果。

---

4. LightGBM 和普通決策樹最大的差別

一棵樹通常不夠強。

例如：

```
Tree 1
  ↓
預測
```

可能預測得不太準。

那我們可以：

> 再蓋一棵樹，專門處理第一棵樹做錯的地方。

於是：

```
Tree 1
   ↓
```

```
第一次預測
   ↓
找出預測錯誤
   ↓
Tree 2
   ↓
修正 Tree 1
   ↓
Tree 3
   ↓
繼續修正
   ↓
Tree 4
   ↓
繼續修正
```

這就是 **Boosting** 的核心想法。

所以你可以把它想成：

> **不是讓一棵超級強的樹解決問題，而是讓很多棵普通的樹，一棵一棵接力，把前面的錯誤修正掉。**

---

5. 為什麼叫 Gradient Boosting？

這裡才開始碰一點數學。

假設真實答案是：

```
100
```

第一棵樹預測：

```
80
```

所以：

```
真實值 - 預測值
= 100 - 80
= 20
```

我們希望下一棵樹：

> 想辦法把這個「20」補回來。

因此：

```
Tree 1 → 預測 80

Tree 2 → 修正 +15

Tree 3 → 修正 +4

Tree 4 → 修正 +1

最後：

80 + 15 + 4 + 1
= 100
```

這就是 Boosting 非常直觀的概念。

---

6. 但真實世界不是這麼簡單

真實資料可能長這樣：

年齡	收入	工作年資	貸款	真實房價
25	40K	2	100K	500
32	60K	5	200K	800
40	90K	10	300K	1200
45	120K	15	400K	1600

第一棵樹會學：

```
收入 > 70K？
```

第二棵樹可能發現：

```
收入高的人裡面：

工作年資 > 10？
```

第三棵樹又發現：

```
貸款金額是不是也很重要？
```

於是最後：

```
模型
=
Tree 1
+
Tree 2
+
Tree 3
+
Tree 4
+
...
+
Tree 100
```

這就是 LightGBM 的基本精神。

---

7. LightGBM 到底比傳統 GBDT 快在哪裡？

這是理解 LightGBM 非常重要的一步。

假設有：

```
1,000,000 筆資料
100 個特徵
```

如果每次建立樹都仔細檢查所有資料和所有切分方式，計算量會非常大。

LightGBM 做了很多工程上的最佳化。

其中兩個很重要：

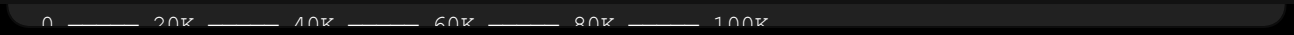
① Histogram-based algorithm

傳統方法可能要仔細尋找：

```
收入 < 50001？
收入 < 50002？
收入 < 50003？
收入 < 50004？
...
```

LightGBM 會把連續數值分成很多個 bucket：

```
收入
```



這樣可以大幅減少需要計算的切分點。

---

② Leaf-wise tree growth

這個概念很重要。

一般傳統做法可能比較像：

```
        Root
       /    \
     A        B
    / \      / \
   C   D    E   F
```

一層一層往下長。

LightGBM 預設比較偏向：

> **哪一個 Leaf 現在最值得繼續切，就優先切哪一個。**

例如：

```
        Root
       /    \
     A        B
    / \
   C   D
      / \
     E   F
```

它會選擇「能讓 loss 降最多」的地方繼續切。

這通常可以讓模型更快降低訓練誤差，但也帶來一個重要風險：

> **比較容易 overfitting（過擬合）。**

所以 LightGBM 的參數設定很重要。

---

8. 現在進入真正的 Python

假設我們已經有資料：

Python 執行

```
import lightgbm as lgb
```

最簡單的分類模型：

Python 執行

```
model = lgb.LGBMClassifier()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

其實只有三件事：

```
X_train
   ↓
model.fit()
   ↓
模型學習
   ↓
model.predict()
   ↓
X_test 的預測結果
```

---

9. X 和 y 是什麼？

這是初學者最容易搞混的地方。

假設資料：

年齡	收入	工作年資	是否買房
25	40K	2	0
32	60K	5	0
40	90K	10	1
45	120K	15	1

我們把：

```
年齡
收入
工作年資
```

叫做 **Features（特徵）**。

所以：

```python
X
```

就是：

```
X =
[
    [25, 40000, 2],
    [32, 60000, 5],
    [40, 90000, 10],
    [45, 120000, 15]
]
```

而：

```
是否買房
```

是我們想預測的答案，也就是：

```python
y
```

所以：

```
y =
[0, 0, 1, 1]
```

可以記成：

> **X = 題目**
>
> **y = 答案**

---

10. 完整的第一個 LightGBM 程式

我們先做一個非常簡單的例子：

Python 執行

```python
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 特徵
X = [
    [25, 40000, 2],
    [32, 60000, 5],
    [40, 90000, 10],
    [45, 120000, 15],
    [28, 50000, 3],
    [38, 85000, 8],
]

# 答案
y = [0, 0, 1, 1, 0, 1]

# 分成訓練資料與測試資料
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# 建立 LightGBM
model = lgb.LGBMClassifier()

# 訓練
model.fit(X_train, y_train)

# 預測
y_pred = model.predict(X_test)

# 評估
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

先不要急著理解每一行。

你現在只需要建立一個概念：

```
資料
 ↓
切成 Train / Test
 ↓
```

```
建立 LightGBM
 ↓
fit()
 ↓
模型學習
 ↓
predict()
 ↓
得到預測
 ↓
評估模型
```

---

11. `fit()` 到底發生了什麼？

這是最值得理解的地方。

當你寫：

Python 執行

```python
model.fit(X_train, y_train)
```

LightGBM 並不是「背答案」。

它會大概進行：

```
Step 1
建立第一棵樹
        ↓
計算目前預測有多差
        ↓
Step 2
尋找一個好的切分方式
        ↓
例如：
收入 > 70,000
        ↓
Step 3
建立下一個 Leaf
        ↓
Step 4
建立下一棵樹
        ↓
修正之前的錯誤
        ↓
Step 5
繼續建立樹
        ↓
```

```
...
        ↓
Step N
得到最終模型
```

所以：

```python
fit()
```

其實代表：

> **「讓 LightGBM 從資料中學習一堆決策樹。」**

---

12. `n_estimators` 是什麼？

例如：

```python
model = lgb.LGBMClassifier(
    n_estimators=100
)
```

可以先非常粗略地理解成：

> 建立大約 100 個 boosting iterations / trees。

例如：

```
n_estimators = 10

Tree 1
Tree 2
Tree 3
...
Tree 10
```

而：

```python
n_estimators=500
```

就是更多輪：

```
Tree 1
Tree 2
...
Tree 500
```

但不是越多越好。

太少：

```
模型學不夠
↓
Underfitting
```

太多：

```
模型過度學習訓練資料
↓
Overfitting
```

---

13. `learning_rate` 又是什麼？

這個參數非常重要。

假設下一棵樹想修正：

```
+20
```

如果：

Python 執行

```
learning_rate=1
```

可能直接：

```
+20
```

但如果：

Python 執行

```
learning_rate=0.1
```

可能只採用：

```
+2
```

也就是：

> **每一棵樹對最終答案的影響程度。**

因此常見搭配是：

```python
n_estimators=500
learning_rate=0.05
```

而不是：

```python
n_estimators=10
learning_rate=1
```

直覺上：

```
learning_rate 小
        ↓
每一步走小一點
        ↓
通常需要更多 trees
```

所以：

```
learning_rate ↓
n_estimators ↑
```

通常會一起考慮。

---

14. `num_leaves` 是什麼？

這是 LightGBM 最重要的參數之一。

一棵樹可以長成：

```
        Root
       /    \
      A      B
```

```
    / \
```

Leaf 就是最下面的節點。

例如：

```python
num_leaves=31
```

代表一棵樹最多可以有 31 個 leaves。

直覺上：

```
num_leaves 小
↓
樹比較簡單
↓
比較不容易 overfitting
```

而：

```
num_leaves 大
↓
樹可以學很複雜的規則
↓
模型能力更強
↓
但比較容易 overfitting
```

所以它非常重要。

---

15. 三個最重要的參數先記住

初學 LightGBM，我建議你先只記：

```python
model = lgb.LGBMClassifier(
    n_estimators=300,
    learning_rate=0.05,
    num_leaves=31
)
```

它們可以粗略理解成：

參數	白話
`n_estimators`	我要蓋幾棵樹
`learning_rate`	每棵樹修正多少
`num_leaves`	每棵樹可以複雜到什麼程度

---

16. 分類和回歸又有什麼不同？

LightGBM 可以處理很多問題。

Classification

例如：

客戶會不會流失？
信用卡會不會違約？
`Email` 是不是垃圾信？

使用：

Python 執行

```python
lgb.LGBMClassifier()
```

Regression

例如：

房價是多少？
銷售量是多少？
明年的營收是多少？

使用：

Python 執行

```python
lgb.LGBMRegressor()
```

例如：

Python 執行

```python
model = lgb.LGBMRegressor()

model.fit(X_train, y_train)

prediction = model.predict(X_test)
```

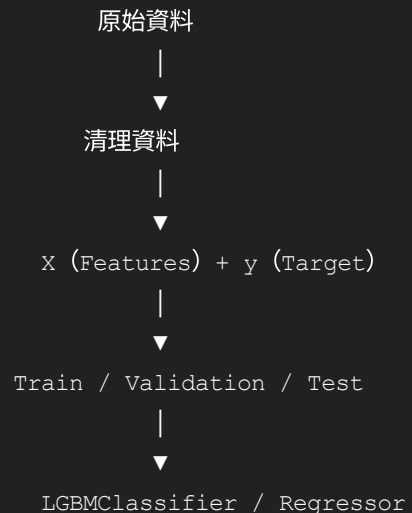
---

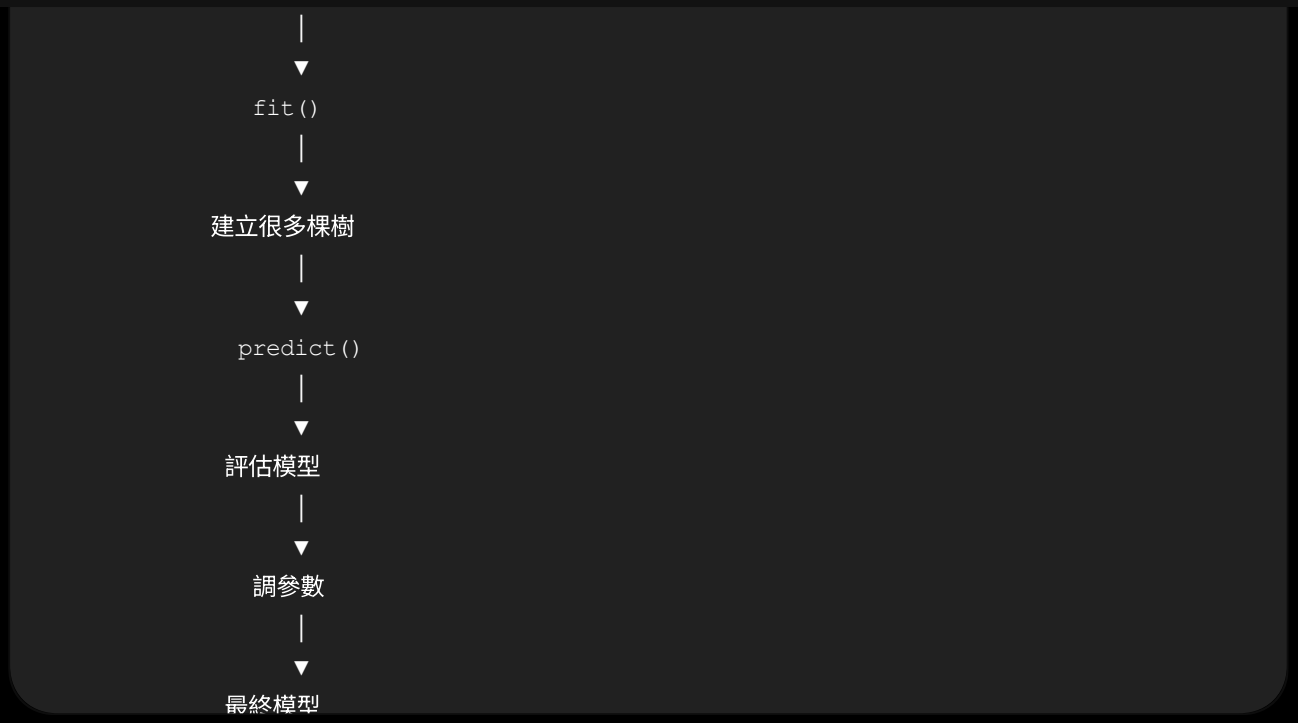
17. 到這裡，你應該已經有一張完整地圖

把整件事情串起來：



而實際使用：





接下來最重要的一步

如果你真的想**從零學會 LightGBM，而不是只會複製程式碼**，我建議我們下一步不要急著學一堆參數。

我可以直接帶你做一個**完整的實戰小專案**，例如：

> **「預測客戶會不會流失」**

我們會從一份 CSV 開始，一步一步實作：

1. `pandas` 讀取資料
2. 看懂每一個欄位
3. 分辨 Feature / Target
4. 處理文字與缺失值
5. Train / Test split
6. 第一次跑 LightGBM
7. 看 Accuracy、Precision、Recall、F1
8. 理解 LightGBM 到底學到了什麼
9. 調 `num_leaves`、`learning_rate`、`n_estimators`
10. 解釋 Feature Importance
11. 最後做出一個可以輸入新資料並預測的模型

這樣你會真正知道**「每一行程式為什麼要寫」**，而不是只記 API。

ChatGPT 可能會出錯。請查核重要資訊。